12 — Neural Networks for Robotics

Robotics and Computer Vision (BPC-PRP)

Course supervisor: Ing. Adam Ligocki, Ph.D.
Ing. Adam Ligocki, Ph.D.

Brno University of Technology
2026

Robotics and AI

Why a Lecture on Neural Networks?

Until now, everything on your robot has been **hand-crafted**:

- IR threshold → "I see the line on the left → turn left."
- Encoder + PID → "speed up the wheel until you reach the target."
- Ultrasonic + rule → "if distance < 0.2 m → stop."

These rules work — until the world becomes too complex to write a rule for.

Neural networks are the standard tool for **learning a mapping** when the rule is too hard to write by hand.

Three typical situations where rules fail:

- **Camera input** — a 640×480 image has 300k pixel values. No rule reliably says "this is a STOP sign".
- **Natural language** — "please drive a bit closer to the wall" has no IF/ELSE.
- **Sensor fusion** — combining lidar + camera + IMU into a single decision quickly explodes in complexity.

In all three cases, a small neural network can outperform months of careful rule engineering.

What Modern Robots Use Neural Networks For

Task	Typical model
Object detection	CNN (YOLO, R-CNN)
Semantic segmentation	CNN (U-Net, Mask R-CNN)
Depth from a single image	CNN (MiDaS, DPT)
Speech to text	Transformer (Whisper)
Voice / text command	LLM (Claude, GPT, Llama)
End-to-end control	Vision-Language-Action (RT-2, OpenVLA)

Today we cover the two **building blocks** under all of these: the CNN and the Transformer-based LLM.

Neural Network Foundations

What Is a Neural Network

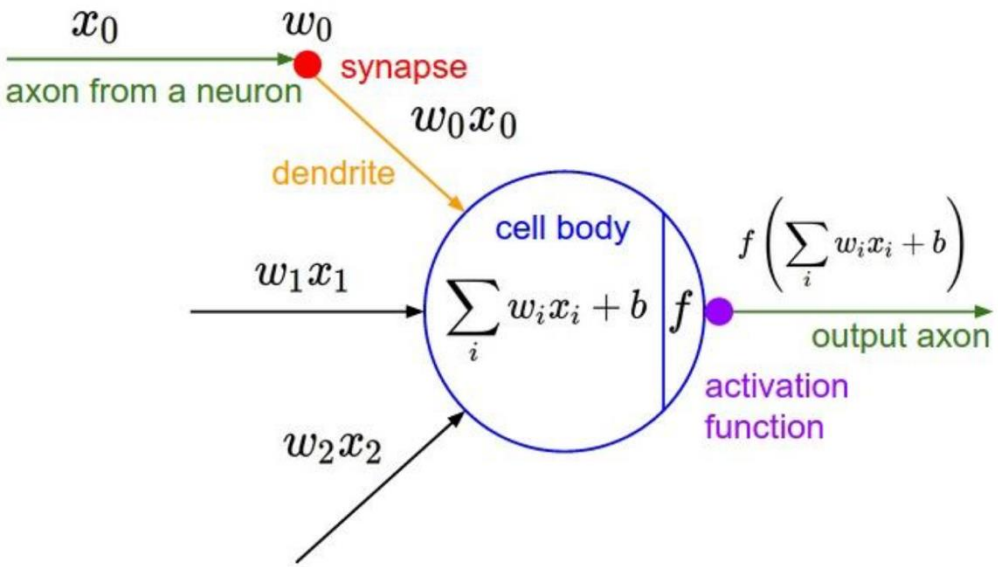
A neural network is a **function** that maps an input vector to an output vector:

f(x; θ) = y

- **x** — the input (e.g. image pixels, token IDs, sensor readings).
- **y** — the output (class probabilities, next token, motor command).
- **θ** — the **parameters** (the numbers we learn from data).

The function is built from a stack of simple blocks. Each block does:

h_next = activation(W · h + b)



https://cs231n.github.io/neural-networks-1/

The Activation Function

Without a non-linearity between layers, the entire stack collapses to a single linear map.

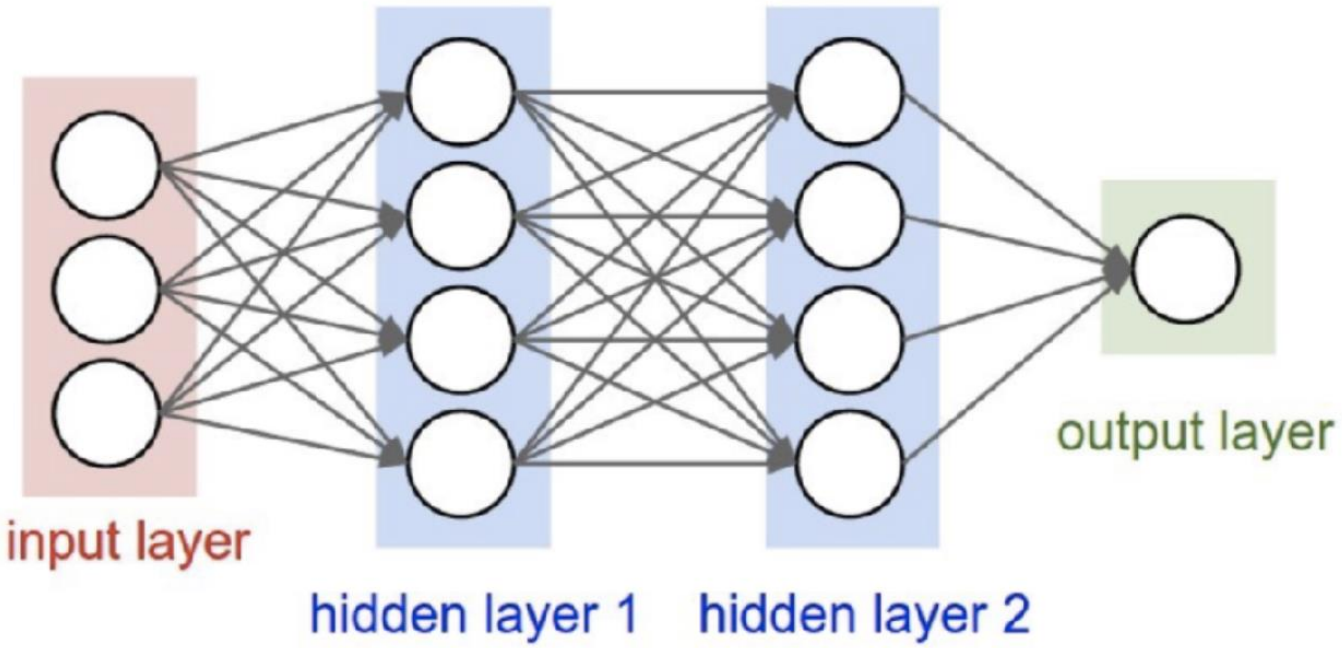
The activation function is what gives the network its expressive power.

Most common choices:

- **ReLU — max(0, x)**. Cheap, fast, doesn't saturate. The default for hidden layers.
- **Sigmoid — 1 / (1 + exp(-x))**. Maps to (0, 1). Used at the output for binary problems.
- **Softmax** — turns a vector of scores into a probability distribution. Used at the output for classification.
  Rule of thumb: **ReLU inside, Softmax at the output.** That covers 90 % of cases.

What Is a Neural Network

$$Y_{N*1} = W_{N*M} * X_{M*1}$$

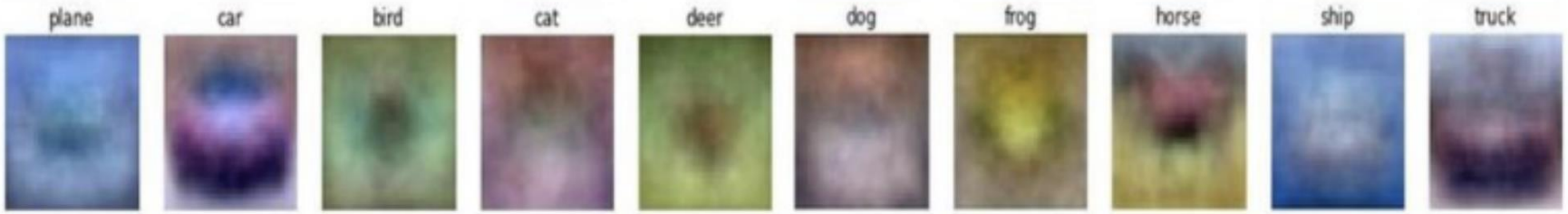


https://cs231n.github.io/neural-networks-1/

What Is a Neural Network

$$f(x_i, W, b) = Wx_i + b$$

Example trained weights of a linear classifier trained on CIFAR-10:



- Slide from: Fei-Fei Li & Justin Johanson & Serena Yeung

How Does the Network Learn?

The training procedure is the same for every neural network — small or large.

1. Forward pass: feed input **x**, compute prediction **y_pred**.

2. Loss: compare to the correct answer with a loss function **L(y_pred, y_true)**.

3. Backward pass: compute the gradient of the loss with respect to every parameter (back-propagation, automatic in PyTorch).

4. Update: step the parameters slightly in the direction that reduces the loss.

5. Repeat for many mini-batches.
This is called **gradient descent**. The same algorithm trains 100 weights or 100 billion.

Train / Validation / Test

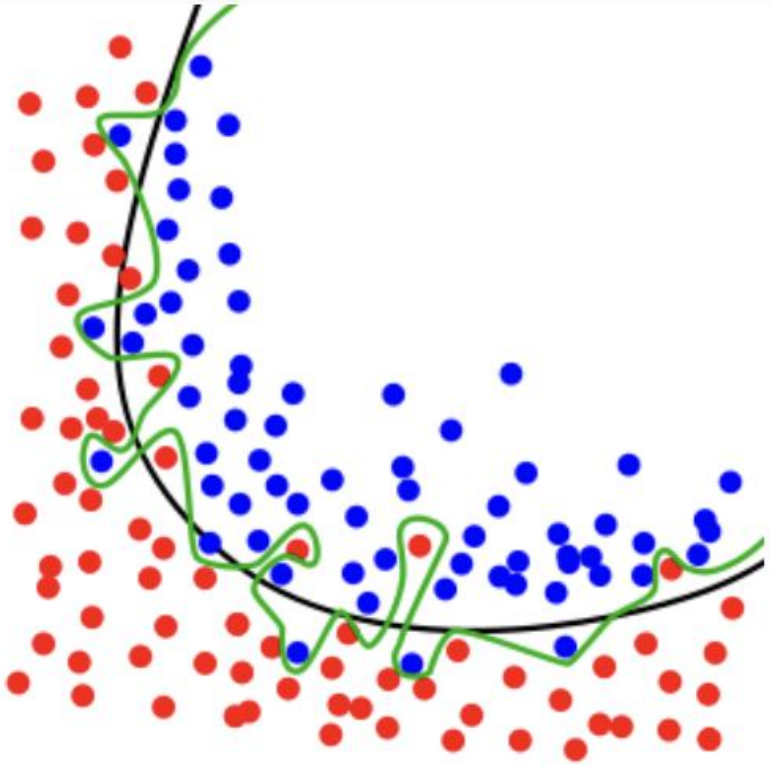
Three **disjoint** datasets, every time:

- **Training set** — what the network actually learns from.
- **Validation set** — checked between epochs to tune the recipe (how many layers, learning rate, …).
- **Test set** — touched **once**, at the very end, to estimate real-world accuracy.

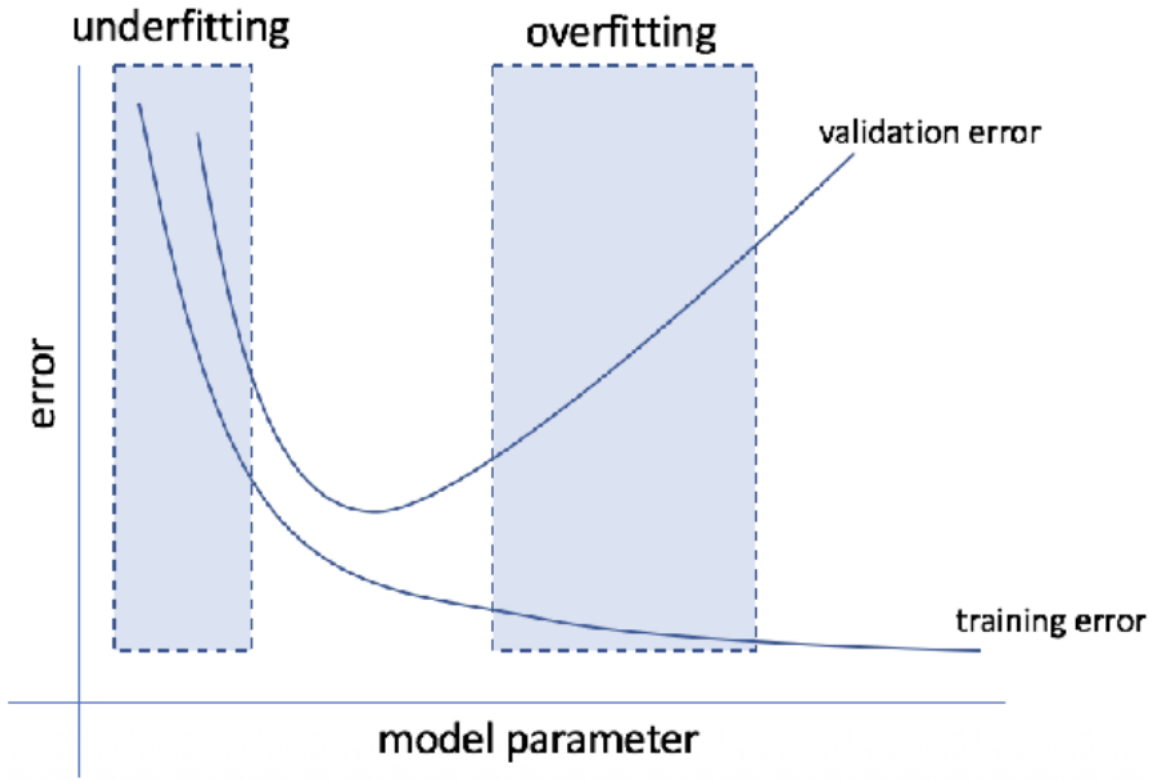
If you tune on the test set you have effectively trained on it. The honest test accuracy is gone.

A typical split is 70 % train, 15 % validation, 15 % test.

Overfitting



https://en.wikipedia.org/wiki/Overfitting



https://www.jeremyjordan.me/evaluating-a-machine-learning-model/

Convolutional Neural Networks

Why Not a Fully-Connected Network for Images?

Imagine a 640×480 RGB image fed into a 1000-neuron hidden layer:

- Input size: 640 × 480 × 3 = **921 600 values**.
- Weights in the first layer: 921 600 × 1000 ≈ **0.9 billion**.

Three problems:

- **Too many weights** — won't fit in memory, won't train.
- **No spatial awareness** — pixel (10, 10) and (10, 11) get unrelated weights.
- **No translation invariance** — shift the line by 2 pixels and every weight has to relearn it.

We need a layer designed for the geometry of an image.

Convolution + ReLU + Pooling

A typical CNN block always has the same three pieces:

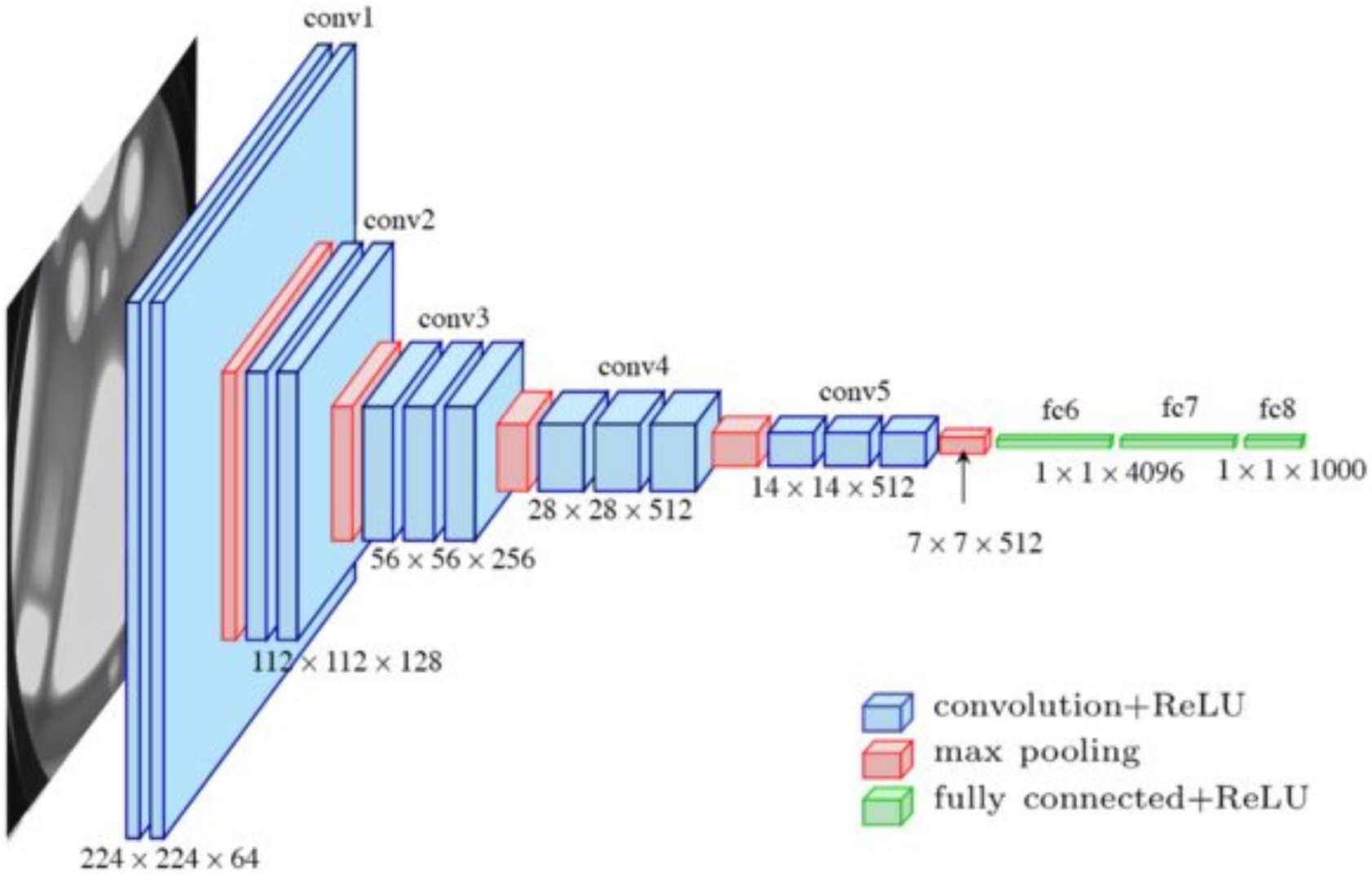
- **Conv2d** — applies several learned filters in parallel. Each filter produces one feature map.
- **ReLU** — keeps only positive activations (the cheap non-linearity).
- **MaxPool 2×2** — keeps only the strongest activation in each 2×2 window. Halves the spatial size.

Stacking 3–5 of these blocks gives a **hierarchy of features**:

- Layer 1 — edges, colour blobs.
- Layer 2–3 — corners, textures, simple shapes.
- Layer 4+ — object parts, complete objects.

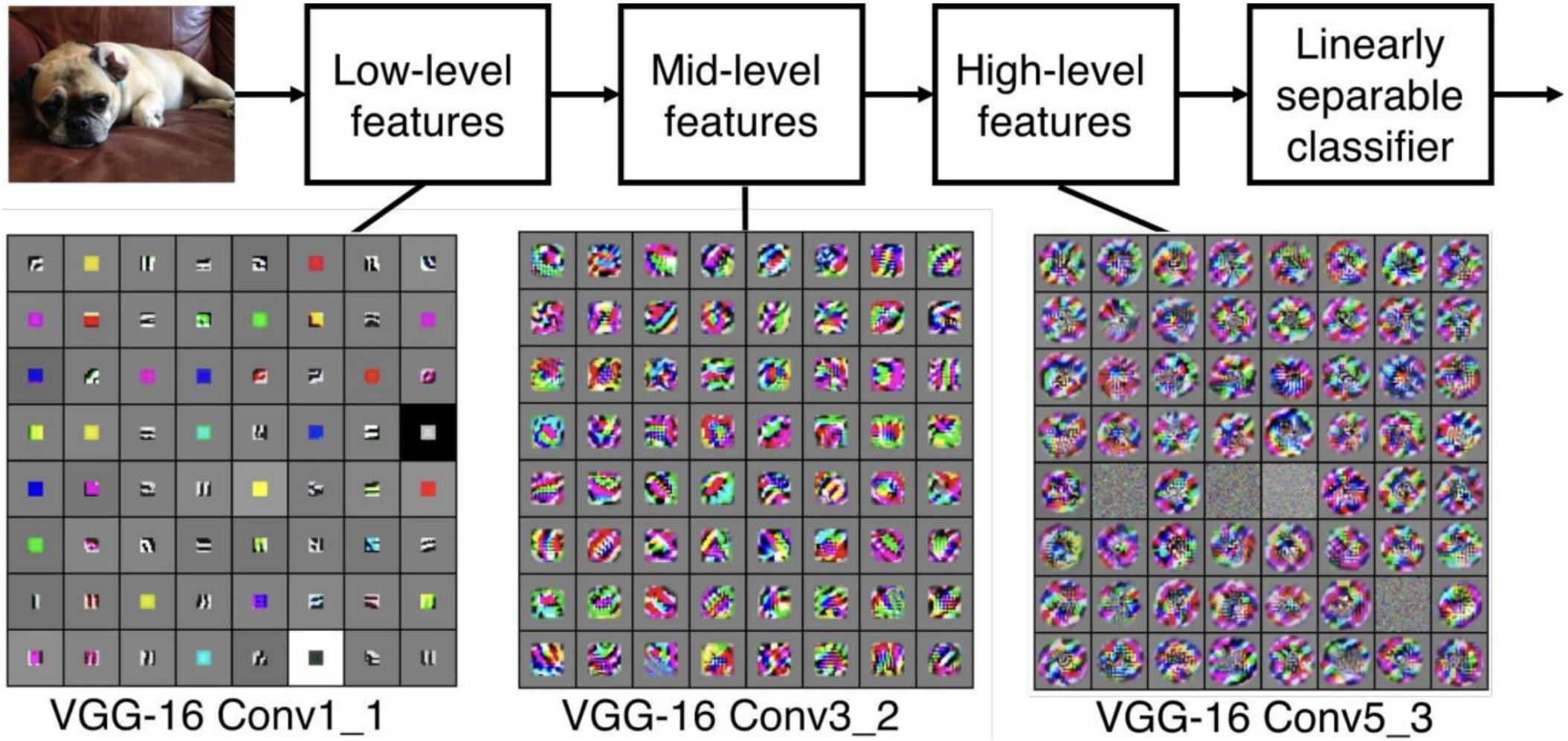
This hierarchy **emerges from training**. Nobody designs the filters.

Convolution + ReLU + Pooling: VGG16 Architecture



Ferguson, M., Ak, R., Lee, Y.T.T. and Law, K.H., 2017, December. Automatic localization of casting defects with convolutional neural networks

Convolution + ReLU + Pooling: VGG16 Architecture



https://stackoverflow.com/questions/48220598/convolution-layer-in-cnn/48280069

Live Demo — the Whole CNN Pipeline in the Browser

The app has five tabs that walk through the entire workflow:

- **Dataset** — train, validation and test sets, class distribution.
- **Model** — architecture, parameter count.
- **Training** — live loss and accuracy curves while the network trains.
- **Evaluation** — confusion matrix, per-class accuracy, sample predictions.
- **Features** — the learned 3×3 filters, and what each layer "sees" on one image.

Large Language Models

What an LLM Actually Does

A Large Language Model is, fundamentally, a **next-token predictor**.

Given a sequence of tokens **t1, t2, …, tk**, it estimates a probability distribution:
p(t_{k+1} | t1, t2, …, tk)

That's it. **Generation is just sampling from this distribution and feeding the new token back in.**

Properties like answering questions, writing code, or following instructions all emerge from this single objective when the model and the data are large enough.

Step One — Tokenization

An LLM does not see characters. It sees **tokens** — sub-word pieces from a vocabulary of ~50 000 entries.

The tokenizer (Byte-Pair Encoding) was learned once, on a huge corpus, to compress text well.

Examples:

- "The quick brown fox" → **[The, _quick, _brown, _fox]** — 4 tokens
- "transformersrock" → **[transform, ers, rock]** — 3 tokens
- "robotický" → **[robot, ick, ý]** — 3 tokens

Common words become a single token. Rare words and other-language words split into many. **Everything inside the model — input, output, attention — operates on token IDs.**

Before Transformers — Recurrent Networks

Before 2017, the standard tool for sequences was the **Recurrent Neural Network (RNN)** and its more capable cousin the **LSTM** (Long Short-Term Memory).

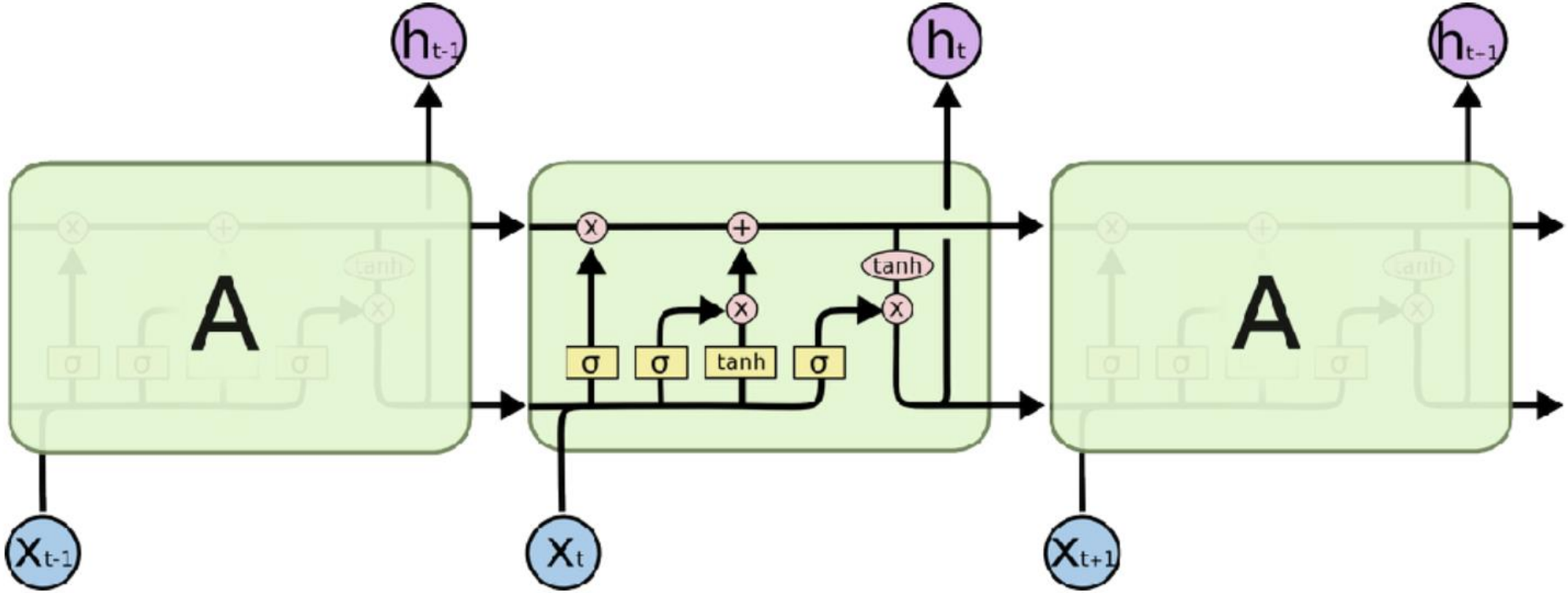
The idea: read the sequence one token at a time, maintain a **hidden state** that summarises everything seen so far.

Two fundamental problems:

- **Sequential** — you must process tokens one after another. Cannot parallelise → slow training.
- **Forgets** — early tokens are diluted through many state updates. Long-range context is lost.

These limits are what stopped RNN/LSTM models from scaling beyond a few hundred million parameters.

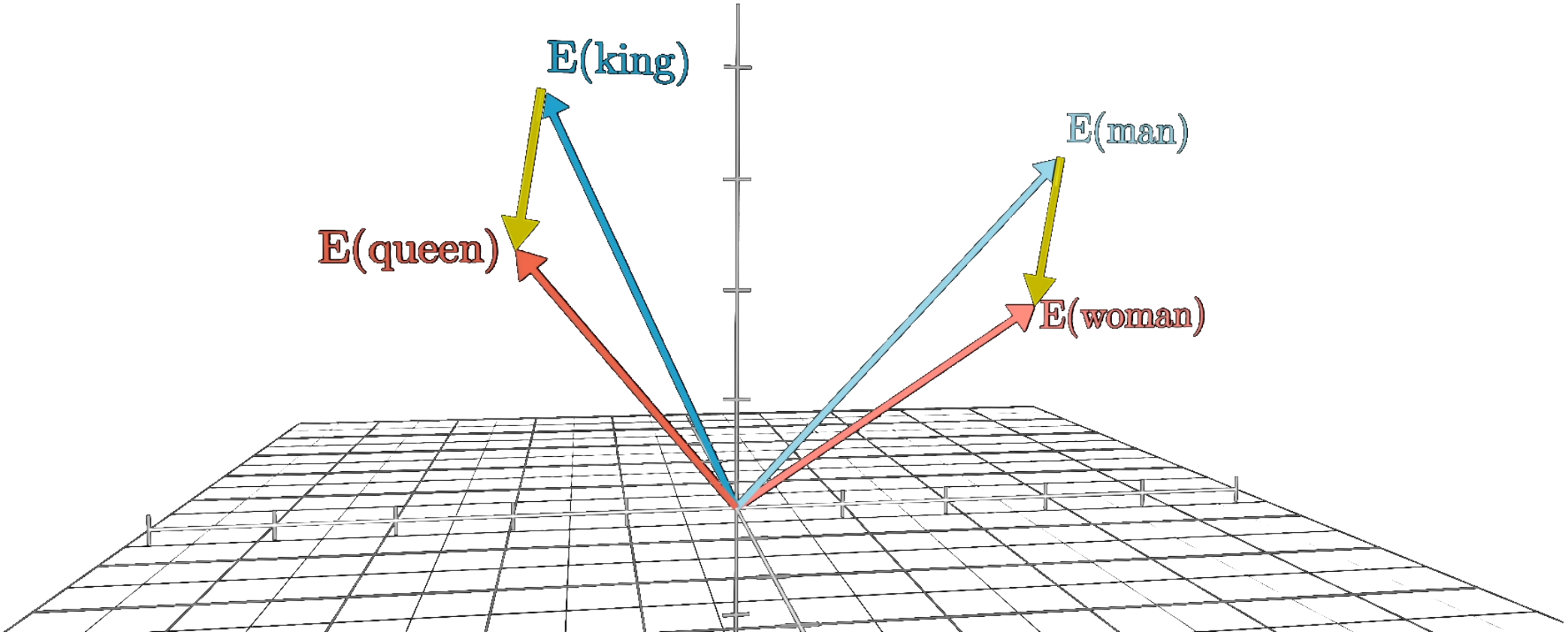
Before Transformers — Recurrent Networks



https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Embedding

$$E(\text{queen}) \approx E(\text{king}) + E(\text{woman}) - E(\text{man})$$



https://www.3blue1brown.com

The 2017 Breakthrough — The Transformer

A paper from Google: "Attention Is All You Need".

The key idea: **throw away recurrence. Replace it with a single operation called self-attention** that lets every token directly look at every other token in the sequence — in parallel.

Why it changed everything:

- **Parallelisable** — all tokens processed at the same time → train on GPUs efficiently.
- **No forgetting** — every token has direct access to every other token.
- **Scales beautifully** — performance keeps improving as you add data and parameters.

Three years later this gave us GPT-3 (175 B params). Five years later, ChatGPT.

Today **every** state-of-the-art language model — GPT, Claude, Llama, Gemini, Mistral — is a Transformer.

Attention — the Intuition

Imagine reading the sentence:

*"The robot picked up the red ball because **it** was light."*

To understand what **"it"** refers to, your eyes naturally jump back to **"ball"** — not "robot", not "red". You give the word "ball" high attention weight and the others low weight.

Self-attention does exactly this, mathematically:

- For each token in the sequence, look at every other token.
- Compute a weight for each — "how relevant is this other token to me?"
- Take a weighted sum of their representations.

The result is a new representation for each token that is **enriched with relevant context** from the rest of the sentence.

Attention — the Mechanism (Q, K, V)

Each token produces three learned vectors:

- **Query (Q)** — what am I looking for?
- **Key (K)** — what do I contain?
- **Value (V)** — what do I actually carry?

For token i, attention compares its query with every other token's key, and uses the matches to mix all the values:

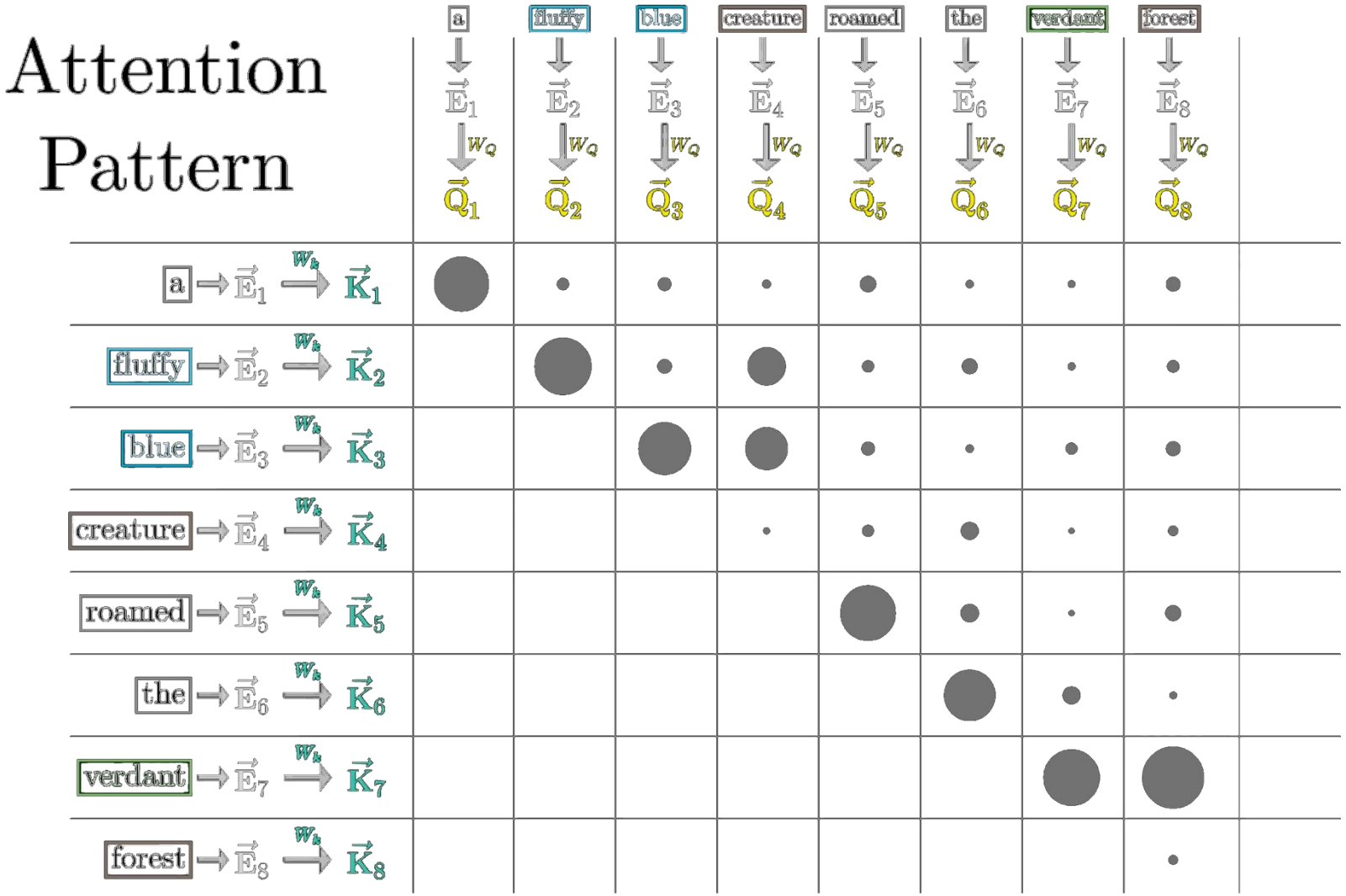
Attention(Q, K, V) = softmax(Q · K^T) · V … (ignoring **sqrt(d)**)

Two practical refinements:

- **Multi-head attention** — do the whole operation several times in parallel with different learned projections; each "head" can specialize.

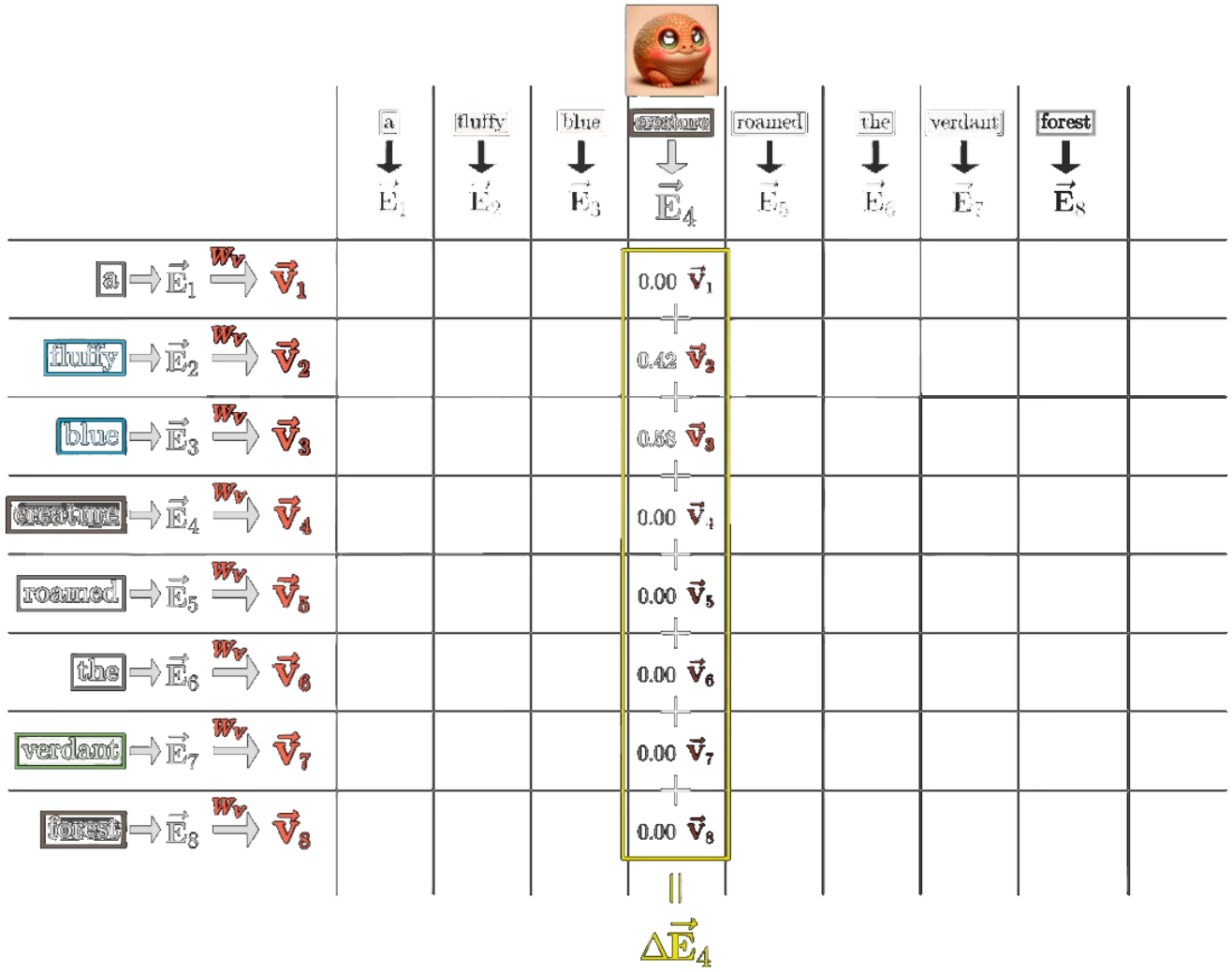
- **Causal mask** — in a generative model, token i may only attend to tokens before it; otherwise we'd be cheating during training.

Attention — the Intuition



https://www.3blue1brown.com

Attention — the Intuition



Two Families — Encoder (BERT) vs Decoder (GPT)

The same Transformer block can be wired up two different ways:

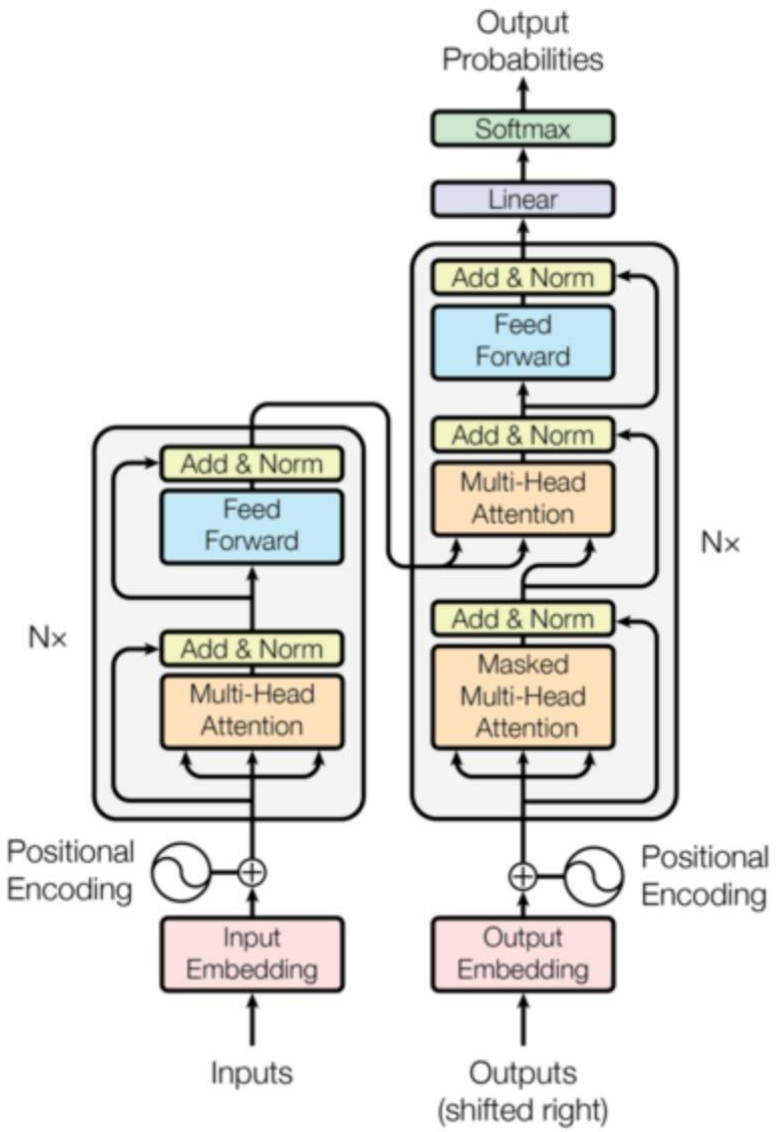
BERT (Bidirectional Encoder, 2018)

- Every token can attend to **every other token, in both directions**.
- Trained by masking random tokens and predicting them.
- Great for understanding text: classification, sentiment, search, embeddings.

GPT (Decoder-only, 2018→today)

- Each token can attend **only to previous tokens** (causal mask).
- Trained to predict the next token, left-to-right.
- Great for generating text: chatbots, code, translation, agents.
  Today's "LLMs" — GPT, Claude, Llama, Mistral — are all **decoder-only**. BERT-style encoders still power search engines and embedding models.

Two Families — Encoder (BERT) vs Decoder (GPT)



https://arxiv.org/pdf/1706.03762

How an LLM Is Trained

A modern LLM is trained in **three stages**, in order:

1. Pre-training — predict the next token on **trillions of tokens** of internet text. Months on thousands of GPUs. This is where 99 % of the cost goes.

2. Supervised fine-tuning (SFT) — fine-tune on curated **(prompt, ideal answer)** pairs. Teaches format and tone.

3. RLHF / DPO — humans rank pairs of replies; the model is nudged toward the preferred one. This is what makes it helpful and aligned.
You and I never do step 1. In practice we **prompt** the model and occasionally **fine-tune** with LoRA on top.

Live Demo — Inside a Real Llama-Style LLM

Default model: **SmolLM2-135M** — a real Llama-style architecture (RMSNorm, RoPE, SwiGLU, multi-head attention), about 270 MB. Runs on CPU.

Five tabs:

- **Tokenize** — your text turned into coloured token chips.
- **Architecture** — layers, attention heads, KV heads, head dimension, parameter count.
- **Next-token** — the top-N candidate next tokens with their probabilities.
- **Generate** — autoregressive sampling with temperature, top-k, top-p.
- **Attention** — per-layer, per-head attention heatmaps over your text.

Integration with ROS2

The Pattern — Same Shape for Both Models

Two reusable nodes, the same shape:

- **CNN node — /camera/image_raw** → preprocess → ONNX → **/cmd_vel**
- **LLM node — /voice_command** (text) → LLM → JSON → **/goal_pose**

Train the model on the laptop.
Export to **ONNX**.
Run inference on the Raspberry Pi with **onnxruntime** — no PyTorch on the robot.

The Pattern — Same Shape for Both Models

```
import rclpy, numpy as np, onnxruntime as ort
from rclpy.node import Node
from sensor_msgs.msg import Image
from geometry_msgs.msg import Twist
from cv_bridge import CvBridge
import cv2

class LineFollower(Node):
    def __init__(self):
        super().__init__("line_follower")
        self.sess = ort.InferenceSession("line_cnn.onnx")
        self.bridge = CvBridge()
        self.pub = self.create_publisher(Twist, "/cmd_vel", 10)
        self.create_subscription(Image, "/camera/image_raw", self.cb, 10)

    def cb(self, msg):
        img = self.bridge.imgmsg_to_cv2(msg, "mono8")
        x = cv2.resize(img, (64, 64)).astype(np.float32)[None, None] / 255
        cls = int(self.sess.run(None, {"input": x})[0].argmax())
        t = Twist()
        t.linear.x = 0.15
        t.angular.z = [+0.8, 0.0, -0.8][cls]   # 0=LEFT, 1=STRAIGHT, 2=RIGHT
        self.pub.publish(t)
```

The Pattern — Same Shape for Both Models

```
import json, rclpy, numpy as np, onnxruntime as ort
from rclpy.node import Node
from std_msgs.msg import String
from geometry_msgs.msg import PoseStamped
from transformers import AutoTokenizer

class TextToGoal(Node):
    def __init__(self):
        super().__init__("text_to_goal")
        self.tok = AutoTokenizer.from_pretrained("HuggingFaceTB/SmolLM2-135M")
        self.sess = ort.InferenceSession("smol.onnx")
        self.pub = self.create_publisher(PoseStamped, "/goal_pose", 10)
        self.create_subscription(String, "/command", self.cb, 10)
    def cb(self, msg):
        prompt = f'Return JSON {{"x":float,"y":float}} for: "{msg.data}"\n'
        ids = self.tok(prompt, return_tensors="np").input_ids.astype(np.int64)
        for _ in range(30):                          # greedy decode
            logits = self.sess.run(None, {"input_ids": ids})[0]
            ids = np.concatenate([ids, [[logits[0, -1].argmax()]]], axis=1)
        text = self.tok.decode(ids[0])[len(prompt):]
        xy = json.loads(text[text.find("{"): text.find("}") + 1])

        goal = PoseStamped()
        goal.header.frame_id = "map"
        goal.header.stamp = self.get_clock().now().to_msg()
        goal.pose.position.x, goal.pose.position.y = xy["x"], xy["y"]
        goal.pose.orientation.w = 1.0
        self.pub.publish(goal)

rclpy.init(); rclpy.spin(TextToGoal())
```

Where to Go Next

Going Further

If you liked **CNNs**, the natural next steps are:

- **YOLO** — real-time object detection.
- **U-Net** — semantic segmentation (pixel-wise classification).
- **MiDaS** — depth estimation from a single image.

If you liked **LLMs**, the natural next steps are:

- **Tool use / function calling** — let the model invoke ROS services.
- **ReAct agents** — reasoning + acting loops.
- **LoRA fine-tuning** — adapt a small LLM to your domain on a single GPU.

References

- **PyTorch documentation** — pytorch.org/docs/stable
- **ONNX Runtime** — onnxruntime.ai
- **HuggingFace Transformers** — huggingface.co/docs/transformers
- **Andrej Karpathy — "Zero to Hero"** — karpathy.ai/zero-to-hero (the best free deep-learning course on the internet)
- **3Blue1Brown — Neural Network series** — 3blue1brown.com/topics/neural-networks
- **Anthropic API documentation** — docs.anthropic.com

Code for today's lecture:
keynotes/2026/bpc-prp-12-nns/ — slides, CNN demo, LLM demo.

Adam Ligocki

adam.ligocki@vutbr.cz

Brno University of Technology
Faculty of Electrical Engineering and Communication
Department of Control and Instrumentation

Robotics and AI
Research Group